

Traveling Salesman Problem - Assignment 2

Keerthi Revanth Kumar

June 24, 2026

1 Methodology and Configurations

1.1 OR-Tools Configuration (Task 1)

The Google OR-Tools `routing` library (Version 9.15.6755) was utilized in this task. Since the OR-Tools engine requires integers, the OSRM distance matrix was multiplied by 1,000 and passed to the routing model. Distances were converted back to kilometers upon output.

The configurations of search parameters were as follows:

- **First-Solution Strategy:** `PATH_CHEAPEST_ARC` was used to construct a strong initial feasible route. This strategy builds the tour by greedily connecting the nearest unvisited nodes.
- **Local-Search Metaheuristic:** `GUIDED_LOCAL_SEARCH` (GLS) was applied to optimize the initial route.
- **Time Limit:** of 30 seconds per instance was enforced.

1.2 Gurobi MIP Start Construction (Task 2)

To warm-start the MTZ model, the completed route generated by OR-Tools was injected as a starting solution. The MIP start was constructed by:

1. Initializing all binary decision variables ($x_{i,j}$) to 0.
2. Iterating through the sequential tour returned by OR-Tools and turning the specific $x_{i,j}$ arcs that belong that tour to 1.
3. Assigning the position index of each city in the tour to the continuous MTZ sequence variables (u_i).

1.3 K-Nearest-Neighbour Sparsification and Symmetrisation (Task 3)

1. **K-NN Selection:** For each city i , the K closest neighbours were identified using `numpy.argsort()` on the distance matrix, and the directed arc (i, j) was added to the candidate set.
2. **Symmetrisation:** To prevent dead ends and guarantee in/out-degree feasibility, the graph was symmetrised. For every directed arc (i, j) generated by the K-NN logic, the reverse arc (j, i) was explicitly added to the set.
3. **Depot Protection:** The Depot node (City 0) was guaranteed valid incoming and outgoing paths by explicitly adding and symmetrising its K closest neighbours, preventing geographic isolation.

The Gurobi variables and constraints were then instantiated *only* over this restricted candidate arc set.

2 Results

Table 1: Google OR-Tools (heuristic)

N (cities)	Total Dist (km)	Runtime (s)	Solver Status	Notes
10	6176.00	30.00	Optimal/Success	PATH_CHEAPEST_ARC GLS +
50	11745.00	30.00	Optimal/Success	PATH_CHEAPEST_ARC GLS +
100	16979.00	30.00	Optimal/Success	PATH_CHEAPEST_ARC GLS +
250	26972.00	30.00	Optimal/Success	PATH_CHEAPEST_ARC GLS +
500	37151.00	30.00	Optimal/Success	PATH_CHEAPEST_ARC GLS +
1000	51967.00	30.00	Heuristic (Limit)	PATH_CHEAPEST_ARC GLS +

Table 2: Gurobi MTZ: cold start vs warm start (OR-Tools MIP start)

N	Cold RT (s)	Cold Gap (%)	Warm RT (s)	Warm Gap (%)	Best Dist (km)	Notes
10	0.01	0.00%	0.01	0.00%	6176.00	Both optimal
50	53.54	0.00%	36.13	0.01%	11745.00	Both optimal
100	1800.03	0.31%	1800.09	0.26%	16818.00	C:TL, W:TL
250	1803.34	18.07%	1800.12	8.37%	25796.00	C:TL, W:TL
500	1801.94	18.64%	1801.51	8.34%	36367.00	C:TL, W:TL
1000	1801.23	71.86%	1801.63	10.99%	51117.00	C:TL, W:TL

TL = Time-limited, C = Cold Start, W = Warm Start

Table 3: K-nearest-neighbour sparsification, N = 500

K	Arcs ($\approx$)	Dist (km)	Runtime (s)	Gap (%)	Status	Notes
10	6178	35487.00	1800.26	3.12%	Time-limited	Heuristic found route
20	12402	35588.00	1800.04	3.63%	Time-limited	Heuristic found route
50	30730	35887.00	1803.89	5.50%	Time-limited	Heuristic found route

3 Analysis and Insights

Heuristic vs. Exact Modeling

Google OR-Tools proved to be really efficient. For smaller instances ($N \leq 50$), the heuristic engine instantly matched the exact optimal distance verified by Gurobi. As N expands beyond 100 cities, achieving exact optimality becomes computationally impractical within a standard time limit. At $N = 1000$, OR-Tools generated a highly viable 51,967 km route in just 30 seconds, demonstrating the benefit of using metaheuristics in large-scale industry standard routing.

The Effect of Warm-Starting

Supplying Gurobi solver with a heuristic incumbent dramatically altered its performance profile.

- **Runtime Reduction:** For $N = 50$, injecting the MIP start allowed Gurobi to skip the early search phases, reducing the time to prove optimality from 53 seconds to 36 seconds.
- **MIP Gap Closure:** For massive instances ($N = 1000$), both runs hit the 30-minute timeout. However, the warm-started MIP gap was exponentially smaller (10.99%) compared to the cold start (71.86%).

The improvement is primarily driven by the *incumbent* (objective value) provided by OR-Tools, which allows Gurobi to aggressively prune branches. However, the weak linear programming relaxation inherent to the MTZ formulation means the theoretical lower bound still moves slowly, preventing the gap from fully closing on large networks.

The Effect of K-NN Sparsification

Restricting the candidate arcs via symmetric K-NN logic successfully mitigated the massive memory and variable overhead for $N = 500$.

- For $K = 10$ the candidate arcs are roughly 6,178. Because the search space was so small, Gurobi explored it efficiently, yielding the shortest total distance (35,487 km) and tightest gap (3.12%) among the three runs before hitting the timeout.
- As K increased to 50, the candidate arcs expanded to 30,730. This slowed Gurobi’s ability to find better incumbent routes before the 1800-second limit, resulting in a slightly longer distance (35,887 km) and a wider gap (5.50%).
- This highlights a critical trade-off in graph sparsification: a tight, feasible K yields better empirical results in a time-limited setting by keeping the variable count computationally manageable.

Overall Trade-offs

- **Heuristics :** Ideal for dynamic, daily logistics where execution speed is paramount and a "good enough" approximation is acceptable.
- **Warm-Started Exact:** Optimal for scenarios requiring mathematically rigorous, near-perfect routing, provided there is sufficient computing time to let the exact solver leverage the heuristic head start.
- **Sparsified Exact:** Necessary for modeling geographically massive networks under memory constraints, trading absolute mathematical certainty (optimal value) for a drastically reduced variable count and stable runtimes.

4 AI Usage Note

For this assignment, i have used Gemini, Chatgpt LLM strictly as an educational and coding assistant. Specifically, AI was used to understand the theoretical mechanics and assist in formulating the syntax for the Gurobi and OR-Tools Python scripts, and format the output logs into tabular LaTeX structures.